



Full length article

BACAD: AI-based framework for detecting vertical broken access control attacks

Ahmed Anas^{a,*}, Ayman A. Alhelbawy^b, Salwa El Gamal^a, Basheer Youssef^a^a Faculty of Computers and Artificial Intelligence, Cairo University, 5 Dr. Ahmed Zewail Street, Giza, 12613, Orman, Egypt^b Faculty of Computers and artificial intelligence, Fayoum University, Fayoum, Egypt

ARTICLE INFO

Keywords:

Broken access control
Attack detector
Vertical access control vulnerabilities
Vertical access control exploitation
Vertical access control attacks
Machine learning
Artificial intelligence
Web application security
Logical vulnerabilities
Exploit detection

ABSTRACT

Vertical Broken Access Control (VBAC) vulnerability is one of the most commonly identified issues in web applications, posing significant risks. Consequently, addressing this pervasive threat is crucial for ensuring system confidentiality and integrity. Broken access control attack detector (BACAD) is a novel framework that leverages advanced AI techniques to neutralize VBAC exploits and attacks in real-time using a dynamic and practical technique. The detection process consists of two steps. The first step is user role classification using an advanced artificial intelligence (AI) model created in a learning phase. The learning phase includes BACAD initial configuration and application user roles traffic generation used for AI model training. The AI model at the core of BACAD analyzes web requests and responses utilizing a robust feature extraction, and dynamic hyperparameter tuning to ensure optimal performance across diverse scenarios. The second step is the decision step, which determines whether the incoming request–response pair is benign or an attack by validating it vs the BACAD session information set. The evaluation against a spectrum of real-world and demonstration web applications highlights remarkable efficiency in detecting VBAC exploits, providing robust application protection against different sets of VBAC attacks. Furthermore, it shows that BACAD addresses the VBAC problem by presenting an applicable, dynamic, flexible, and technology-independent solution to counter VBAC vulnerability risks. Thus, BACAD contributes significantly to the ongoing efforts aimed at enhancing web application security.

1. Introduction

Web applications, which are software programs running on web servers and accessible through clients like browsers, mobile, or desktop apps, have become essential in today's world. They serve as a primary channel for accessing a wide range of services such as banking, governmental, shopping, and social networking. However, these applications and their programming interfaces (APIs) are vulnerable to various types of attacks due to their inherent structure, which exposes them to external sources and networks [1]. This susceptibility makes it crucial to implement robust countermeasures, as web applications are often targeted by malicious actors exploiting vulnerabilities arising from inadequate adherence to secure coding and architectural best practices, especially since they frequently contain sensitive data and are used to run critical business systems.

There are two primary categories of vulnerabilities in web applications: technical vulnerabilities and logical vulnerabilities, with broken access control vulnerabilities being a major concern within the

latter category. [2–6]. Technical vulnerabilities, often stemming from inadequate input validation, enable attackers to conduct attacks like cross-site scripting and injection vulnerabilities such as SQL Injection by injecting malicious payloads into the application to perform unauthorized actions. Technologies like web application firewalls (WAF) offer some protection against these attacks [7].

Logical vulnerabilities, especially those related to broken access control, present a significant and complex challenge. Despite their importance, research and technical solutions have made limited progress in addressing these issues compared to advancements targeting technical vulnerabilities. Logical vulnerabilities occur when attackers bypass intended rights and permissions, resulting in unauthorized access, privilege escalation, information leakage, and data integrity breaches. These vulnerabilities often stem from insecure coding practices and weaknesses in session management, authentication, authorization, and access control mechanisms.

The Open Web Application Security Project (OWASP) reports that 94% of tested applications exhibit broken access control vulnerabilities,

* Corresponding author.

E-mail addresses: ahmed.anas@fci-cu.edu.eg (A. Anas), aah03@fayoum.edu.eg (A.A. Alhelbawy), s.elgamal@fci-cu.edu.eg (S.E. Gamal), basheer@fci-cu.edu.eg (B. Youssef).<https://doi.org/10.1016/j.eij.2024.100571>

Received 8 July 2024; Received in revised form 22 October 2024; Accepted 27 October 2024

Available online 10 November 2024

1110-8665/© 2024 The Authors. Published by Elsevier B.V. on behalf of Faculty of Computers and Artificial Intelligence, Cairo University. This is an open access article under the CC BY-NC-ND license (<http://creativecommons.org/licenses/by-nc-nd/4.0/>).

making it the most commonly identified issue in penetration tests. Notably, broken access control has risen from fifth place on the OWASP Top 10 web application vulnerabilities list in 2017 to the top position in 2021 [8]. This alarming trend highlights the limitations of traditional detection methods, which fail to adapt to the dynamic nature of modern web applications in terms of efficiency, practicality and scalability.

The increasing scale of web applications, combined with the severe consequences of unauthorized access, underscores the need for robust approaches that effectively mitigate risks without requiring extensive manual configuration or reliance on specific technologies. This highlights the critical necessity for comprehensive protection against broken access control attacks.

Access controls in general can be categorized into the following types: Vertical Access Control, Horizontal Access Control, and Context-Dependent Access Control. Vertical access controls are mechanisms that restrict access to specific functions to distinct types of users or user groups within an application. These controls offer a detailed implementation of security measures intended to uphold organizational policies like the separation of duties and the principle of least privilege. Vertical access control vulnerabilities can be categorized into multiple buckets that can lead to vertical privilege escalation: Privilege Escalation, Forcible Browsing, Sensitive Function Broken Access Control, User Role Controlled by Request Parameter, Broken Access Control Resulting from Platform Misconfiguration, Broken Access Control Resulting from HTTP Method, Broken Access Control Resulting from URL-Matching Discrepancies, and Broken Access Control Resulting from Insecure Default Settings.

Privilege Escalation occurs when an application lacks safeguards for its high-privileged roles access. For example, administrative functions supposed to be accessible only from an administrator's private page or portal (e.g., http://example.com/admin_mgmt) might be accessible from a standard user's page by directly navigating to a specific admin URL. Forcible Browsing occurs when an attacker forces browsing to pages that require authentication for unauthenticated users or allows access to a highly privileged page from users with inadequate rights. Vulnerabilities occur when some private pages do not have proper authentication and authorization policies. Sensitive Function Broken Access Control arises when certain business functions, designed to be accessible only by users with specific roles, can be executed by users of a different role due to the application's implementation. User Role Controlled by Request Parameter occurs when the user role is defined based on user-controllable data items such as parameter values or cookies. For example, an attacker can change the role parameter from 'user' to 'admin' in a URL to gain access to admin functions. Broken Access Control Resulting from Platform Misconfiguration occurs when access control is enforced through platform roles such as configuration files or HTTP headers. Broken Access Control Resulting from HTTP Method occurs when an application neglects to properly restrict access to specific HTTP methods (such as PUT or DELETE), allowing an attacker to exploit these methods. Broken Access Control Resulting from URL-Matching Discrepancies occur when controls fail to properly validate capitalization or overlook URL mapping considerations. Broken Access Control Resulting from Insecure Default Settings occurs from accidentally leaving a debug mode enabled in the production environment, granting unintended access to sensitive data or functions.

Other main types of access controls are Horizontal and Context-dependent access controls. Horizontal access controls are designed to limit access to resources and data objects specifically to authorized users. Horizontal privilege escalation occurs when a user gains unauthorized access to data objects or resources that belong to another user, which they are not supposed to access based on their privileges, role, or group. Context-dependent access controls restrict the availability of features and resources according to the state of the application or the user's interaction with it. These safeguards ensure that users are unable to invoke functions out of order. For example, after a customer's

payment has been processed on a retail website, they may not be able to change the contents of their shopping cart.

The problem of detecting and preventing broken access control (BAC) attacks resides in the need for a thorough comprehension of the functionality intended for an application versus its actual implementation. This necessitates a thorough examination and modeling of the input-output relationships as well as the anticipated data flow. To defend against BAC attacks, several research directions have been developed, as elaborated in previously published survey research paper [9]. These include improving the software development life cycle and using development frameworks, analyzing both dynamic and static code, and developing black-box approaches to detect and prevent attacks. However, the solutions currently in place, as detailed in the related work section, have several shortcomings that prevent them from being widely implemented to protect real-world applications.

This paper presents a novel framework for detecting and addressing vertical broken access control vulnerabilities. The primary contributions are as follows:

- Design of the BACAD (Broken Access Control Attack Detector) framework, which is capable of dynamically identifying and mitigating vertical broken access control attacks. BACAD is practical and scalable, making it suitable for deployment in real-world applications. It operates independently of the underlying technology or programming languages, requiring no access to source code. It ensures minimal overhead throughout the entire traffic journey and does not require manual intervention enhancing its practicality and ease of deployment. Moreover, it offers customizable responses to attack detection to mitigate risks.
- Design of AI-Driven detection of VBAC attacks by learning and building profiles of different user roles without the need to visit every system page, effectively reducing false positives associated with unlearned pages.
- Design of dynamic machine learning model selection to choose the most effective model for optimal detection and prevention of broken access control attacks.

This paper is structured as follows: Section 2, titled "Related Work" provides an overview of existing research relevant to this study. In Section 3, titled "Proposed Framework" the proposed framework is detailed, covering the architecture, design and methodology of BACAD. Section 4 describes the AI model. Section 5, titled "Results Evaluation and Discussion" presents the outcomes of the model along with a comprehensive analysis and discussion of these results. Section 6 titled "Conclusion" summarizes the research findings and their implications. Finally, Section 7 titled "Future Work" outlines potential avenues for further investigation and development in this field.

2. Related work

Research on mitigating risks related to broken access control vulnerabilities, specifically the vertical type, can be categorized into three main areas. The first area emphasizes preventing the emergence of such vulnerabilities during the coding process, as well as identifying and rectifying them through code reviews. The second area is dedicated to identifying vulnerabilities within existing running applications and taking necessary actions to mitigate them. The third area focuses on the detection and prevention of the exploitation of existing broken access control vulnerabilities in operating applications (e.g., in [10], and [11]).

The literature emphasizes the significance of securing web applications from logic flaws throughout the secure software life cycle stages, proposing a range of solutions including the adoption of secure coding practices during the development stage (e.g., in [12], and [13]) in addition to the use of secure web frameworks (e.g., in [14–17], and [18]) that offer a base for implementing security policies during development and architectural phases, as well as enhancing auditing

and authorization [19], and [20]. Moreover, The role of the source code review stage is highlighted as significant added value for detecting security flaws including broken access control (BAC) vulnerabilities.

Literature surveys as in [21–25], and [26] show that security testing is presented as a practical approach for uncovering vulnerabilities by analyzing the behavior of running systems and the implementation of access control policies. Dynamic analysis solutions such as DetLogic [22], and LogicScope [23], along with similar models, aim to detect vulnerabilities in running applications.

Researchers emphasize the advantages of employing security testing tools and automation, noting that these tools deliver thorough evaluations and help identify potential gaps that may be missed during manual testing. This, in turn, enhances the detection of Broken Access Control (BAC) vulnerabilities and bolsters overall application security. Studies [27–29], and [30] pointed out the value of integrating security testing tools with code review and threat modeling in identifying various security issue categories.

Research directions and approaches involving frameworks and coding, including source code review methodologies and tools, are often seen as inadequate in protecting against BAC attacks [31]. This is because they need to be implemented early in development and cannot be applied to running applications. Dynamic analysis shares limitations with source code analysis or monitoring, focusing more on identifying vulnerabilities than preventing exploitation. Both approaches require access to the source code for refactoring, which poses a challenge since many applications are closed-source, and owners may be reluctant to share it, as further discussed in the survey by Almushiti et al. (2024) [10]. There is also no guarantee that issues are addressed promptly by closed-source providers.

Finding vulnerabilities demands resources and time, burdening developers and potentially giving attackers an advantage since the application continues running in its vulnerable state, posing a risk. Moreover, these approaches depend heavily on specific technologies and programming languages, making them less effective in diverse environments. They also suffer from high false positive rates. Dynamic analysis, in particular, depends on thorough application crawling by the assessor in a learning phase, leaving unexplored areas unprotected.

Limited research has been conducted on preventing the exploitation of broken access control vulnerabilities in applications during runtime, focusing on exploit and attack prevention rather than detecting the vulnerabilities. This approach aims to mitigate some of the previously mentioned limitations. The research in this area has utilized various techniques and methodologies to differentiate between benign and malicious user interactions within applications. Nevertheless, the existing body of literature presents several significant shortcomings, rendering it challenging to implement in real-world applications. These issues are further detailed in subsequent sections.

Researchers presented solutions that are tailored to a specific domain or have a limited scope, such as Invariant Detector (IVD) [32] designed for online social networks (OSNs), and InteGuard [33] which is limited to the protection of the third-party services APIs integration. Additionally, IVD has specific constraints as it requires application-based customization and rule adjustments. Moreover, it only covers the integrity factor, which makes it challenging to implement widely and scale to other real-world applications. PHP-Sensor [34] is designed to identify attacks relevant to specific workflows by extracting and applying a unique set of axioms. These axioms evaluate HTTP requests and responses in real-time; non-adherence to the expected flow model or corresponding axiom results in flagging as a workflow violation attack. However, PHP-Sensor faces several limitations. It is technology-dependent, offering protection only for PHP applications, and does not support AJAX, which is commonly used in modern applications. Additionally, it struggles with complex workflow scenarios and introduces a significant performance overhead. Solutions like Double Guard [35] and TamperProof [36] have been introduced to provide BAC attack

prevention solutions. However, these solutions have crucial limitations that make them impractical for real-world deployment, along with other specific drawbacks. For instance, Double Guard utilizes a container-based web server architecture where each session is assigned to a separate web server, isolated from other sessions, and mapped to a single user. This enables the separation of different information flows by session and allows linking each session to its corresponding database query or request. However, this solution faces significant challenges in scalability and performance management because the proposed architecture requires a container image for each user session. Real-life applications can exceed billions of sessions daily, making this approach impractical. Additionally, Double Guard identifies user sessions by their IP addresses, which is unreliable for users who use proxies (e.g., corporations or countries) and mobile users who dynamically change their IP addresses. The TamperProof method involves generating a concealed field with an ID to impose constraints on JavaScript and HTML. However, TamperProof is unable to protect modern applications that utilize AJAX or dynamically alter client-side code. Moreover, web pages that use HTML and JavaScript obfuscation present an additional challenge for the TamperProof technique. Frameworks such as BLOCK (BLACK-box approach for detecting state violation attacks) [37] and SENTINEL (SEcuriNg daTabase from logIc flaws iN wEb appLIcation) [31] offer approaches designed to act as source code black-box solutions. BLOCK employs a proxy-based approach, wherein it assesses requests and responses in real-time using invariants identified from the sequences of web requests/responses and the associated values of session variables extracted from server-side files and session databases under normal system operations. A potential state violation attack occurs when there is a deviation from these invariants in either the request or response. On the other hand, SENTINEL aims to address two shortcomings in BLOCK, which lacks visibility into indirect relationships. To do this, SENTINEL is designed to be positioned inline to extract application SQL queries and responses nevertheless, SENTINEL also requires access to the application server side to map and extract session variable values from the server side. A significant limitation of both solutions is their treatment of all unexplored paths or documented invariants as potential attacks and blocking them, contributing to an increase in the rate of false positives. Moreover, any modifications in application structure or data layer invalidate the previously learned invariants, necessitating a new learning cycle to mitigate false positives and prevent incorrect blocking of the application on a wide scale. Additionally, the integration needed for gathering session variable data is highly dependent on the technology used, which makes it hard to scale and deploy. SENTINEL in particular faces further constraints due to database query technology and performance overhead. The multiple constraints and limitations in BLOCK and SENTINEL related to high dependency on applications technology, deep integration, and learning variants model inflexibility make it impractical to deploy in the real world.

Solutions that depend on access to source code have been introduced to protect against BAC vulnerabilities. For example, Swaddler [38] is designed to detect context-dependent attacks by mapping the code execution paths in PHP-based applications. This mapping is linked to session variables, among other characteristics, during the learning phase and aims to detect anomalies in defensive mode. However, this solution is limited by design due to its dependency on technology and the requirement for source code access, and it does not address VBAC attacks.

The taxonomy and shortcomings of the related work are summarized in Table 1, which highlights various existing limitations and provides a detailed comparison of data sources and domain constraints. Additionally, it elaborates on several key aspects as follows: BAC Privilege Escalation Detection (PED), Dependency on Development Languages and Platforms (DLPI), Source Code Required (SCR), Blocking Mode (BM) shows whether the solution can operate in a mode that not only detects but also actively blocks attacks, rather than solely functioning in detection mode without blocking capabilities, False Positives

Table 1
Related work limitations comparison.

Solution	Source	Domain	PED	DLPI	SCR	BM	FP	MIR	MCR	PO
BLOCK [37]	Session Variables, Requests, and Responses	General	Yes	No	No	Yes	High	Yes	Yes	Median
SENTINEL [31]	SQL Queries and Responses, and Session Variables	General	Yes	No	No	Yes	Exist	Yes	Yes	Degraded
IVD [32]	Database Requests	Domain: OSN. Enforce Integrity No Confidentiality	Yes	No	No	Yes	Exist	Yes	Yes	Degraded
InteGuard [33]	Integrator, and Provider Traffic	Third-party APIs Merchant side only	No	Yes	No	Yes	Exist	Yes	Yes	Median
Swaddler [38]	Code Paths and Session Variables	Workflow Attacks only	No	No	Yes	No	High	Yes	Yes	Degraded
Double Guard [35]	User Requests and Database Requests	Identifies Sessions by IP	Yes	Yes	No	No	Exist	Yes	Yes	Deficient
TamperProof [36]	Form Analysis, and Form Constraints	No AJAX NO/Web 2.0/3.0 (Legacy Applications)	Yes	No	No	Yes	High	No	No	Degraded
PHP-Sensor [34]	HTTP Sequences and Session Variables	No AJAX Support (Legacy Applications)	No	NO	Yes	Yes	Exist	Yes	Yes	Degraded

(FP) frequency, Manual Intervention Required (MIR) and custom roles creation for false positive handling, Manual Crawling Required (MCR) for each application function and page, where any non-manually visited and learned functions are considered false positive violation, and finally Performance Overhead (PO).

In addition to focusing on mitigating Broken Access Control (BAC) vulnerabilities, various studies have emphasized the importance of addressing broader security practices that influence the effectiveness of access control measures. Research highlights that insufficiently defined access roles and permissions can contribute to privilege escalation, which increases the risk of BAC exploitation. Another critical aspect is the configuration and enforcement of security policies, where misconfigurations or outdated policies can unintentionally allow unauthorized access. Moreover, studies have shown that inadequate security awareness and training among developers and administrators can lead to improper implementation of access controls, further compounding the issue (e.g., [39–41], and [42]).

While there have been various efforts to detect and mitigate Broken Access Control (BAC) vulnerabilities through different methodologies, the existing approaches still face notable challenges, particularly in their applicability to real-world applications. Many of these solutions are limited by their heavily dependence on source code access, the need for extensive manual intervention, and high false-positive rates, which complicate their implementation in dynamic and diverse environments. Furthermore, the existing solutions often struggle with scalability and performance issues, which hinders their effectiveness in addressing vulnerabilities during runtime. Given these shortcomings, there is a pressing need for an innovative solution that not only overcomes the limitations of current methodologies but also enhances the overall security posture of applications.

The proposed BACAD framework aims to respond to this need by leveraging artificial intelligence to detect Vertical Broken Access Control (VBAC). It achieves this by learning and building profiles of different user roles without the need to visit every system page, thereby reducing the incidence of false positives among unlearned pages, which solving that severe limitation in other finite state machine and graph-based solutions. Furthermore, the framework is independent of the underlying technology or programming languages used. It eliminates the requirement for source code access, ensuring its practicality and ease of deployment with minimal overhead throughout the entire traffic journey. The solution does not require manual intervention or custom

rules to be explicitly created for any specific application. Additionally, it offers the flexibility to provide a custom course of actions based on attack detection, such as only monitoring, issuing alerts, dropping the HTTP response, or executing specific procedures to mitigate the risk.

3. BACAD framework architecture

BACAD, designed to identify VBAC attacks, operates in two primary phases, the Training phase and the Detection phase. Fig. 1 presents a high-level architecture diagram illustrating the components involved in both phases. In the training phase, the system operator generates initial training data comprised of HTTP Responses by traversing application roles and assigning role types for each set of user traffic within a sanitized environment. Upon completion of this step by the system operator, the training data are forwarded to the machine learning model to extract essential features and learn a set of models, as elaborated in Section 4. In the detection phase, inference models are employed to assess every web request and response pair, identifying any deviations or attacks.

The framework encompasses various integrated components collaborating to achieve the designated protective goal. Different integrated components utilized during the learning phase, including the Configuration Module, Training Module, Dataset, and Models. The required parameters of the initial configuration for the new application were derived from walkthroughs that generated annotated traffic for each system user role. Man-In-The-Middle Proxy (Mitmproxy) is a versatile component for intercepting and modifying HTTP traffic. In its reverse proxy mode, it serves as an intermediary between clients and servers. Leveraging Mitmproxy functionality enables overriding the request and response handlers, allowing for necessary modifications to integrate different framework components as follows. The preprocessing function is responsible for extracting request and response bodies along with data encoding and cleansing prior to the feature extraction component. Moreover, it enables the AI model construction phase, which forms the foundation for the inference engine used for user identification in attack detection.

The process for completing the learning phase while onboarding the new application involves several steps, as illustrated in Fig. 2. First, the initial configuration of the application is performed, which includes defining the application name. Second, the operator specifies the number of user roles needed within the application, reflecting the different

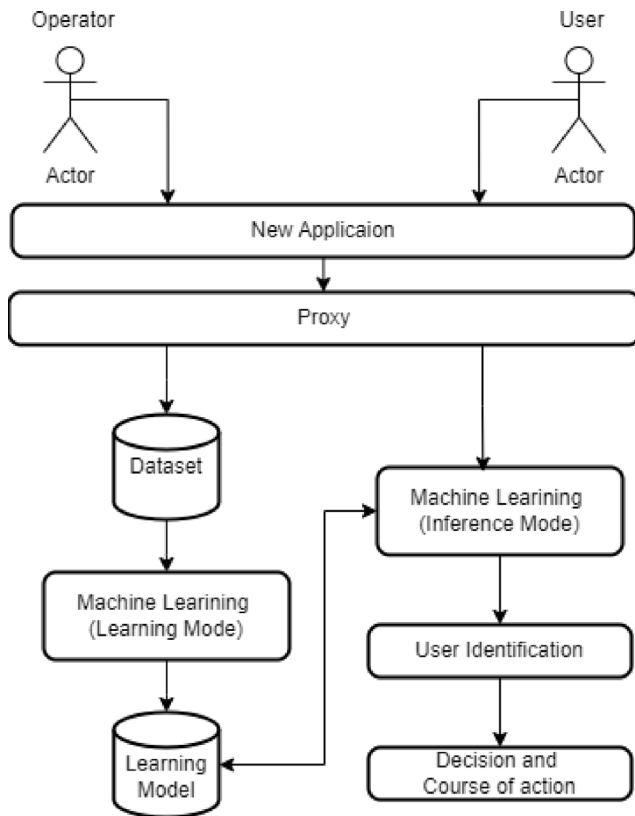


Fig. 1. BACAD high-level architecture.

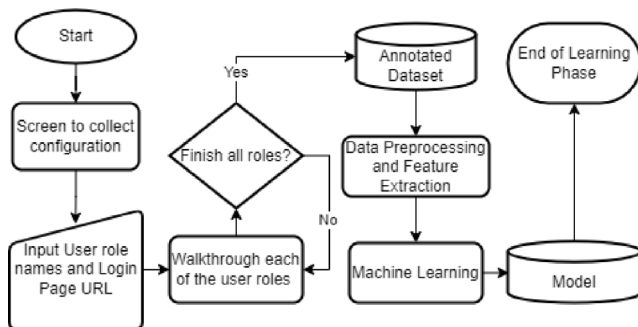


Fig. 2. BACAD learning phase flowchart.

roles of users (e.g., unauthenticated users, authenticated users, and admin users). Third, the operator identify the post-login page, as well as the logout page (e.g., logout.php). Fourth, the operator is expected to conduct a walkthrough to explore the various role pages, services, and functions within each application role scope. Therefore, Resulted HTTP responses are saved by the proxy alongside with corresponding user role. Finally, the feature extraction function processes each HTTP response, generating a complete annotated dataset that serves as input to the machine learning classifier module for AI model creation.

The detection phase is composed of multiple integrated components designed to provide both inference functions and response actions. In the detection module, the user's traffic is directed to the Mimtpoxy Proxy Module, which in turn allows traffic to be processed by the preprocessor module. The preprocessor module's function is to clean and encode the data using the same process defined in the learning phase. The encoded data is then utilized by the user identification ML models module. ML module is responsible for identifying the user type based on the web response, utilizing the previously created model in

learning mode. Finally, the action module is responsible for analyzing the identified user value versus the application status and context to take the required action. The action module can be extended to either give an alert or block the responses or also can be integrated with the application to initiate a course of action or rollback scenario.

The detection phase is a crucial process, acting as the core component of the system. The process which is elaborated in Fig. 3 can be described as follows. Once the system loads, it creates a custom empty session table. This table is pivotal for user role tracking moving forward. The BACAD system involvement starts once the user interacts with the application as the proxy intercepts the traffic (HTTP Requests/Responses). BACAD performs two initial validations, checking for its cookies in the request against the session table corresponding data to decide the proper system course of action, after that, it checks if the incoming request is a Logoff request that has been identified earlier in the learning phase. In case no valid cookies were identified indicating that this user has not been previously classified, the BACAD Framework generates a BACAD session ID and adds it to the session table. It also sets the corresponding cookie value in the user's browser as part of the HTTP response through the set-cookie header. At the same time, within the session table, the BACAD process designates this session as an unauthenticated user session (for instance, labeled as 'A', where 'A' denotes the least privileged or unauthenticated role) until further classification occurs during the login process, as explained further. In case the request was identified to Logoff page, BACAD process designates this session property as an unauthenticated session as well.

The system feeds the HTTP response to the feature extraction function to generate a response feature vector. Next, the BACAD AI model, which has already been trained during the learning phase, classifies the user role based on the extracted response feature vector. At this particular point, there is an important checkpoint that identifies the execution flow direction. Two directions are available, direction one which in case the request is on the login page, direction two is for any other pages. In case login-specific request/response pairs direction, BACAD updates roles based on the proposed classification of the login response (i.e. if the response resulted from the request page is for authenticated user B, The session table entry for this session is updated to role B.). In case of any other non-login requests are sent, principal validation takes place as follows. If the model proposed classification of the request-response pair is a higher role than the current user session role, the system considers this request-response pair a potential attack. Then, It executes a proper course of actions such as alerting, blocking the response, or sending specific calls to the application for rollback scenarios if needed.

4. Artificial intelligence model

The artificial Intelligence model is a critical component of BACAD framework for detecting VBAC attacks. BACAD framework operates in two main phases i.e. learning and detection phases. During the learning phase, initial training data is generated by the system operator who traverses application roles and assigns role types to each user traffic within a sanitized environment. The AI model then processes this data to extract essential features and finalize the learning phase. In detection phase, a trained model is employed to evaluate incoming web requests and responses to identify user roles, subsequently enabling the framework to detect attacks.

The pipeline of the AI model consists of two main steps, Feature Extraction then model learning/inference. The model input is annotated response bodies while the output is the predicted role for each response body. Within the Feature Extraction step, Features were converted into numerical vectors emphasizing key terms, then scaled for standardized values, enhancing model training and inference.

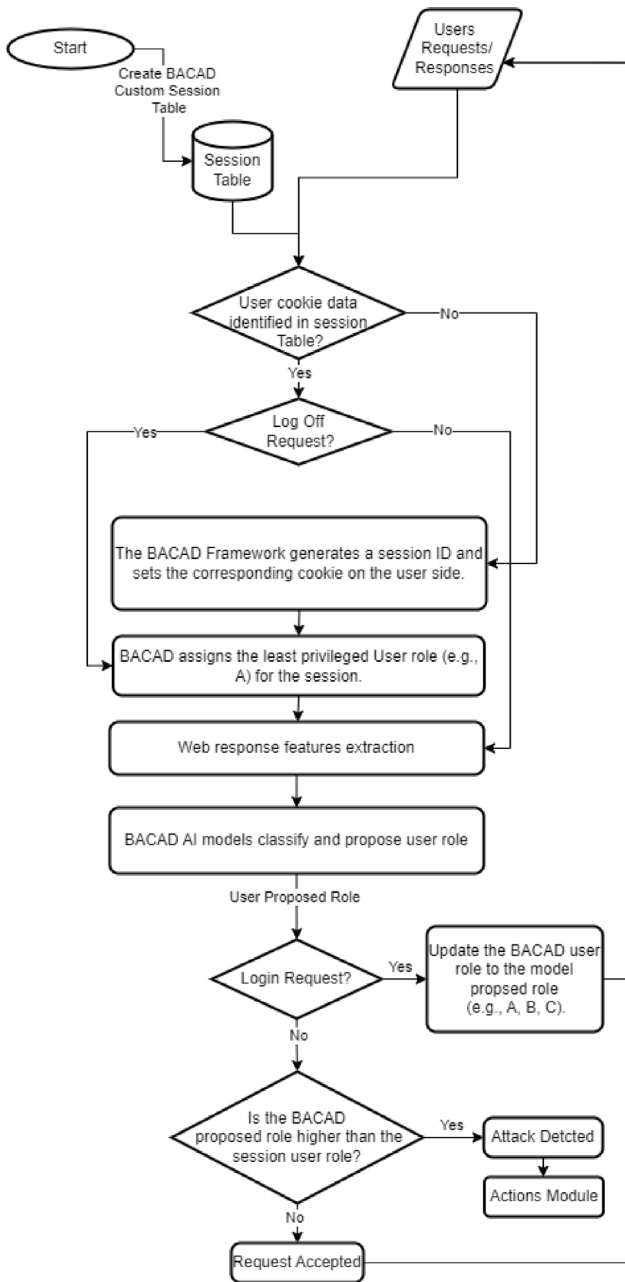


Fig. 3. BACAD detection flowchart.

4.1. Feature extraction

The dataset exhibits a high level of noise due to the presence of misleading information. A feature extraction employs a special pipeline to get a reliable set of bag-of-words (BOW) [43]. Firstly, remove irrelevant entries such as binary content, client-side script files, and CSS files. Secondly, input traffic data files are processed to extract request properties and response bodies, which then are divided into separate files for requests and responses, removing unnecessary or misleading information such as timestamps, custom headers, cookies information, etc. Thirdly, a crucial feature encoding step takes place in which the HTML tags and body text values are tokenized and accumulated into dictionaries with assigned unique codes for each token. These dictionaries are prepared in the learning phase and then used as reference in the inference mode. So, the same dictionaries are used in both training and testing to ensure uniform representation. These dictionaries are then

used to replace tokens with their corresponding unique code values in response files.

In the feature extraction pipeline, two key functions were utilized. First, the Term Frequency-Inverse Document Frequency (TF-IDF) weighting scheme was applied to generate document vectors for each response. This method transforms textual data into numerical vectors by emphasizing terms that are frequent within a specific document but infrequent across the corpus, thereby enhancing the representation of important words. Second, MaxAbsScaler was applied for feature scaling. Created during the training phase and used in inference mode, MaxAbsScaler scales each feature by its maximum absolute value, ensuring that all values fall within the range $[-1, 1]$. The combination of TF-IDF and MaxAbsScaler provides a robust approach for normalizing and preparing feature vectors, facilitating effective model training and consistent inference.

4.2. Model training

BACAD AI model is trained using two different classic machine learning approaches i.e. Support Vector Machine (SVM) [44] and Random Forest (RF) [45] classifiers. The selection of Support SVM and RF classifiers for training the BACAD AI model is particularly apt due to the characteristics of the dataset provided, notably its small size. SVM has been known to perform well in scenarios with limited data [46], as it seeks to find the optimal hyperplane that maximizes the margin between classes, thus mitigating the risk of overfitting even in small datasets. Similarly, Random Forest, through its ensemble learning approach, can effectively handle small datasets by constructing multiple decision trees and aggregating their predictions, thereby reducing the variance associated with individual trees and enhancing generalization [45]. By utilizing SVM and RF, the BACAD AI model can leverage their respective abilities to generalize from limited data while still capturing complex patterns, ensuring robust performance despite the dataset small size.

The SVM classifier is a powerful supervised learning model that finds the optimal hyperplane to maximize the margin between different classes. It supports various kernel functions, including linear, polynomial, and radial basis functions (RBF), to handle non-linear data. In this study, various kernels were explored to determine the best fit for the dataset. Dynamic hyperparameter optimization was performed using cross-validation on a parameter grid that included C, gamma, and kernel to identify the optimal parameters. These hyperparameters were then applied to train the SVM model on the training data.

The RF classifier is an ensemble learning method that constructs multiple decision trees during training. In this study, the RF model was configured with dynamic hyperparameters, including the number of estimators, maximum depth, minimum samples required to split a node, and minimum samples required at a leaf node.

The machine learning model stands as a cornerstone within the BACAD framework, vital for identifying user roles that are integrated to stop Vertical Broken Access Controls (VBAC) attacks. Its seamless integration highlights the framework protection efficiency, ensuring robust defense mechanisms with minimal disruption. Through careful feature extraction and model training, the framework utilizes the power of AI to protect against VBAC attacks effectively.

5. Evaluation results and discussion

A comprehensive evaluation strategy was employed to assess the effectiveness and performance of BACAD framework and the AI models used. The accuracy of the AI models in user identification was assessed separately, and the framework's ability to detect VBAC attacks was also evaluated. Additionally, the performance overhead of BAC-AD and its proxy component was examined throughout the traffic flow.

Both BACAD and its AI models have been assessed across various application types and technologies, ensuring robustness and generalization. Applications are selected carefully to cover a wide spectrum

Table 2
Applications description summary.

App name	Description	Technology	Files count	Vulnerabilities
AltoroJ	Sample Banking Application	J2EE, Apache Derby SQL database, Tomcat web server.	276	Admin Dashboard and Functions VBAC
Google Gruyere	Basic Social Media Network	Python.	23	Server Management Dashboard and functions VBAC
Scarf	Conference Management System	PHP, MySQL.	21	Admin functions VBAC (CVE-2006-5909)
Cyclone Transfers	Sample money transfer service	Ruby, Ruby-On-Rails, SQLite.	205	Personal User Data Catalog VBAC

of business eras, specifically a personal banking platform, social media networks, a money transfer service, and a conference management platform. These applications are built using different technology stacks such as J2EE, Python, Ruby on Rails, and PHP, as well as various database management systems. Also, they are deployed on different web server technologies.

5.1. Test applications and proxy

While the proposed solution is technology-independent, BACAD is evaluated using a set of open-source web applications with a variety of underlying technologies with different application sizes. Four applications are selected to evaluate BACAD namely AltoroJ, Google Gruyere, and Cyclone Transfers. A summary description for each of these applications is shown in Table 2.

AltoroJ is a sample banking J2EE web application, particularly developed in Java and JSP. AltoroJ runs over Tomcat 7 server and it utilizes Apache Derby as its SQL database. It serves as a simple and robust platform for demonstrating real-life application security issues. The application is vulnerable to multiple VBAC vulnerabilities, which allows the malicious normal application non-admin user to access the “/admin/admin.jsp” dashboard page and its functions. These functions include creating new application users, assigning bank accounts to them through the “Add Account” and “Add User” functions, and the ability to change a user’s password.

Google Gruyere emulates the core features of a basic social platform, allowing users to set up a profile with details like name, photo, a highlighted message, and website link. Users can then oversee their profiles and share brief messages, akin to snippets. The code is written in Python relies on templates and uses a simple data storage system implemented in Python. The application is vulnerable to multiple VBAC vulnerabilities, which allow the malicious normal application non-admin user to access the “/manage.gtl” page and its functions including tenant destruction through the “/quitserver” page.

SCARF, Stanford Conference and Research Forum is a conference management platform developed by Stanford University. It assists researchers and conference administrators in creating discussion forums for their papers. The system is PHP-based and uses MySQL as a backend database. However, it possesses an authentication bypass vulnerability (CVE-2006-5909) enabling attackers to access administrative functions directly. The flaw lies in `generaloptions.php`, where it fails to verify if the session variable `$_SESSION['privilege']` matches “admin” before retrieving or updating system settings and user accounts in the database. SCARF application is used in evaluation by different researchers for similar problems (e.g., [31,37], and [47]).

Cyclone Transfers is a sample money transfer service, part of an OWASP project called Broken Web Applications (BWA). Cyclone Transfers consists of multiple common Ruby on Rails vulnerabilities. The application is developed using the Ruby programming language and is based on the Ruby on Rails framework, utilizing an SQLite database. The application is vulnerable to VBAC vulnerabilities that allow an

unauthenticated and unauthorized attacker to access users’ catalog and personal data through the “/users” and “/search?method=get” functions.

Both web applications, SCARF and AltoroJ, have been deployed on a 2.3 GHz Core 8, Windows Machine with 4 GB RAM, running Windows 10 operating system, Tomcat v7.0, XAMP v5.6.3, PHP 5.2.0, and MySQL v5.7. Cyclone Transfers has been deployed on a 2.3 GHz Core 1 Linux server with 1 GB RAM, running Ubuntu 14.04.6 LTS and a bundled WEBrick web server. Google Gruyere has been tested over the internet against sandboxed instances provided by Google running with Python 2.7.

BACAD, including its Mitmproxy component, has been deployed to intercept the traffic between users and the web application. Labeled HTTP traffic corresponding to user roles has been generated for each application to mimic exact user behavior and simulate real user interactions. In the evaluation, fresh traffic was generated to serve as input for the AI model component during testing. Additional traffic was also generated for attack detection evaluation purposes using the same setup, as elaborated later.

5.2. Dataset generation

The dataset generation process involves exporting of traffic intercepted by the proxy, specifically Mitmproxy, during system operator walkthroughs. This traffic represents interactions within the application under study and is captured in a sanitized environment to ensure data integrity and privacy. Given the nature of the problem, the dataset used for training is not large-scale but rather generated by the system operator. During these walkthroughs, the operator annotates the traffic, assigning class labels to different types of interactions based on their understanding of the application’s roles and functionalities. These annotations serve as ground truth labels for supervised learning, enabling the machine learning model to learn from the labeled data.

The walkthrough process involves systematically exploring the application functionalities and user roles, simulating various user interactions such as logging in, accessing different pages or features, and performing actions within the application. As the operator navigates through different interactions ensuring that the model learns from a diverse range of scenarios and behaviors within the application. These logged interactions, along with the operator annotations, form the training dataset for the machine learning model. Table 3 provides a summary of the training dataset per application, outlining the count of requests/responses and user roles for each application under evaluation.

5.3. Evaluation of AI models performance

A separate in-depth analysis and assessment were conducted to evaluate the effectiveness and performance of the AI model, which forms the core component of BACAD framework. Precision, Recall, F1-score, and Accuracy are widely recognized performance metrics in the

Table 3

Training dataset.

Application name	# of Requests/Responses	# of User Roles
AltoroJ	3163	3
Google Gruyere	512	3
Scarf	445	3
Cyclone Transfers	346	2

AI domain. Accordingly, these metrics were utilized to comprehensively assess model performance. Table 4 presents the testing results, detailing Precision, Recall, and F1-score for each user role, along with the overall model Accuracy.

Accuracy measures the overall correctness of the model by calculating the ratio of correctly predicted instances to the total instances. **Precision**, defined as the ratio of true positive predictions to the total positive predictions. **Recall**, defined as the ratio of true positive predictions to all actual positives which assesses the model ability to identify positive instances. Finally, **F1-score** is the harmonic mean of precision and recall, provides a single metric that balances both concerns, making it particularly useful for imbalanced datasets.

The performance of AI models in the BACAD framework was evaluated using four key metrics: Precision, Recall, F1-score, and Accuracy. These metrics are essential for assessing models effectiveness in classifying different user roles which is a core component for identifying potential attacks. From the results presented in Table 4, Distinct performance variations were observed between the SVM and RF models, with SVM consistently outperforming RF across most applications and roles.

Notably, for roles such as AltoroJ's Role B and Cyclone Transfers Role A, both models exhibited high Precision, Recall, and F1-scores, indicating their robustness in predicting true positives without a significant compromise in misclassifications. However, RF struggled in identifying Role B in Google Gruyere, as indicated by its lower Recall (0.47), leading to a suboptimal F1-score (0.59). This disparity in performance, particularly in imbalanced data scenarios, highlights SVM's superior handling of such challenges. The high Accuracy scores, especially in Scarf (1.00 for both models), confirms the overall effectiveness of the AI models in most applications.

5.4. Evaluation of detection performance

The process of assessing the detection performance of BACAD VBAC attacks as a complete solution involves executing real attacks on target web applications. A set of attacks, including vulnerability exploits, were manually executed on each web application. A total of 10 attacks were performed against each of the vulnerabilities described in the applications section, with varied circumstances each time. This included scenarios such as assuming different user identities or executing varied sequences of actions before launching the attack aimed at performing unauthorized access by circumventing vertical access controls. Table 5 provides an overview of the test sets and detection outcomes. It is evident that all attacks were successfully identified, with no instances of false negatives.

5.5. Evaluation of response time and overhead performance

As the BACAD detection system is positioned between the client and web applications, it inevitably impacts the response time of the web application. Firstly, the proxy module intercepts and forwards the messages exchanged between the user and the web application, inherently increasing the response time. Secondly, the integrated detector assesses the web responses, introducing further delay. To quantify the performance overhead introduced by BACAD, a JMeter simulator was employed to execute 1,000 requests per application for each configuration, resulting in a total of 12,000 requests. This included a predefined

sequence of operations, and the response time for each web request was recorded.

Performance is compared across three configurations:

1. Application default in the absence of BACAD or BACAD proxy component (Mitmproxy) to measure average application response time (Application Default).
2. Deployment of BACAD proxy component only (Mitmproxy) to intercept the traffic, without the integrated detection inference engine or other BACAD framework logic.
3. Deployment of the BACAD framework, including its proxy component.

Fig. 4 presents a summary of the average response time for each application under these scenarios. It is evident that the average response time increases by approximately 90 Milliseconds when BACAD is deployed as elaborated excluding proxy overhead. While the resulting response time remains acceptable, it is noteworthy that a big portion of the overhead stems from Mitmproxy which affects the overall overhead. The current prototype implementation does not entail modifications or configurations to optimize Mitmproxy performance or framework pipeline.

5.6. Discussion

Detection results demonstrate the effectiveness of BACAD in identifying VBAC attacks across a range of target web applications. All conducted attacks were successfully detected in all cases, yielding zero false negatives. This is a critical achievement as it suggests that the system is highly reliable in identifying unauthorized access attempts that exploit vertical access control vulnerabilities. Robustness of detection mechanism was tested through 10 varied attacks per vulnerability, yet BACAD consistently produced accurate results, regardless of the attack sequence or the user identity assumed during the tests.

These findings emphasize BACAD ability to adapt to various real-world attack scenarios and configurations, showcasing its reliability and versatility in different application environments. Additionally, the absence of false negatives enhances the confidence in the framework, as it indicates that no VBAC attack scenarios slipped through undetected. Consequently, BACAD proves itself as a reliable detection tool, ensuring that sensitive actions and functions remain safeguarded from unauthorized access.

While results shows that the AI models are effective at detecting user roles, there still a small chance to misidentify user role. This success can be attributed to the integration of the AI models with the BACAD framework advanced logic, which enables efficient and effective attack detection. The combination allows more robust and accurate detection mechanism, leveraging the AI model ability to understand and learn different user roles within the context provided by the framework logic to identify unauthorized access attempts effectively.

Response time evaluation shows that deploying BACAD increases response times, mainly due to the Mitmproxy component used for traffic interception. While this delay is minimal and acceptable, optimizing Mitmproxy or framework pipeline could further reduce overhead. The detection module itself adds little extra delay, suggesting room for improvement without compromising security performance.

The results of this study cannot be directly compared to other approaches because those methods did not utilize AI and relied instead on hand-crafted techniques with numerous custom rules, many of which are highly specific to their respective domains. Additionally, these alternative methods are heavily dependent on specific technologies and development frameworks. Moreover, the corresponding papers often omit numerous crucial details required for reimplementing, such as precise algorithmic steps, datasets, data pre-processing procedures, and parameter settings which complicate any attempts of reproduction to carry out direct comparison.

Table 4
AI models performance results comparison.

App name	Roles	SVM				RF				Support ^a
		Precision	Recall	F1-score	Accuracy	Precision	Recall	F1-score	Accuracy	
AltoroJ	A	0.93	0.95	0.94	0.94	0.93	0.95	0.94	0.89	145
	B	0.97	0.97	0.97		0.88	0.97	0.92		187
	C	0.89	0.85	0.87		0.86	0.66	0.75		95
Google Gruyere	A	0.96	0.90	0.93	0.95	0.95	0.90	0.92	0.82	123
	B	0.89	0.94	0.91		0.80	0.47	0.59		100
	C	0.99	1.00	1.00		0.76	1.00	0.86		148
Cyclone Transfers	A	1.00	0.97	0.99	0.99	1.00	0.97	0.98	0.98	120
	B	0.98	1.00	0.99		0.97	1.00	0.98		122
Scarf	A	1.00	1.00	1.00	1.00	1.00	1.00	1.00	1.00	147
	B	1.00	1.00	1.00		1.00	1.00	1.00		108
	C	1.00	1.00	1.00		1.00	1.00	1.00		126

^a Support: Number of Tested Samples.

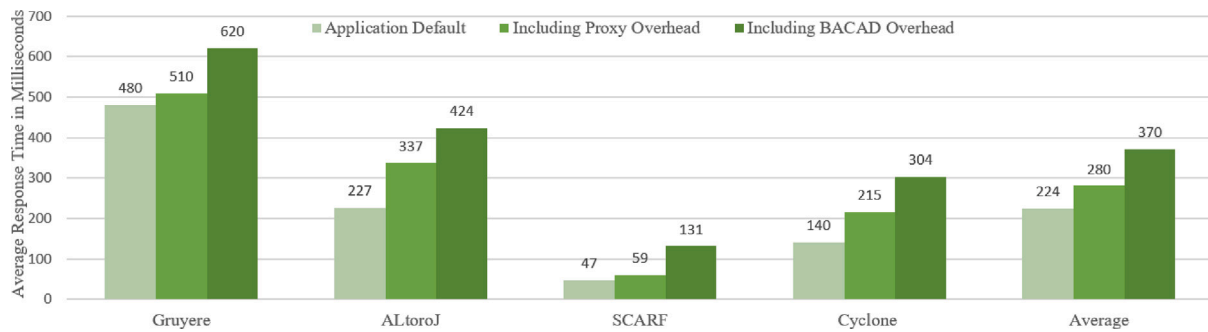


Fig. 4. BACAD average response time.

Table 5
Detection performance against VBAC attacks.

App name	# of Vulnerabilities	# of Attacks	# of Detections
AltoroJ	4	40	40
Google Gruyere	2	20	20
Cyclone Transfers	1	10	10
Scarf	1	10	10

6. Conclusion

Vertical Broken Access Control (VBAC) is considered one of the most critical yet challenging vulnerabilities to fix. The BACAD framework presented in this study offers a robust and dynamic solution for detecting VBAC attacks by integrating a unique artificial intelligence model. The framework employs a practical approach to protection, providing a good balance of cost-effectiveness and efficiency. This strategy for addressing VBAC related attacks is expected to significantly enhance protection efforts. A thorough evaluation was conducted using metrics such as accuracy, precision, recall, and F1-score to assess the AI model's overall classification performance. The results indicate that, given the model parameters and dataset, the SVM algorithm outperformed the Random Forest algorithm. Additionally, simulations of real attacks against a set of applications proved framework reliability and effectiveness with 100% detection rate. Overall, BACAD demonstrates significant potential in mitigating the risks of VBAC attacks and safeguarding web applications from exploitation, ultimately supporting the maintenance of their confidentiality and integrity.

7. Future work

Future work on the BACAD framework will focus on the integration of real-time machine learning techniques. By leveraging online

machine learning algorithms, the framework can adapt to new data and changing environments without a significant initial training phase. This approach utilizes methods such as incremental learning and transfer learning, which allow the model to update its parameters dynamically as new data becomes available. This enhancement is expected to improve the system responsiveness and operational efficiency, facilitating quicker detection of access control violations.

Another area of exploration involves the reformulation of solution approach by utilizing diverse data sources for learning, including operational data and historical data. These sources may provide a comprehensive foundation for training deep learning models and neural networks, which typically require substantial datasets for optimal performance. To support this effort, the framework may consider incorporating techniques such as data augmentation. Data augmentation could create variations of existing data to enhance dataset diversity, moreover, involve generating additional synthetic examples to ensure a robust training process.

Additionally, BACAD framework will focus on extending its capabilities to address a wider range of broken access control vulnerabilities, including horizontal and contextual access control violations. For horizontal access control vulnerabilities, research will investigate methods for detecting manipulation of user identifiers and their associated permissions. In the context of contextual access control, the framework will explore the integration of algorithms that can track multiple user actions and behaviors over time. Techniques such as Long Short-Term Memory (LSTM) networks, Recurrent Neural Network (RNN), and Gated Recurrent Units (GRUs) will be considered to enhance the model ability to analyze user behavior and recognize potential exploitation patterns. This multidimensional approach aims to strengthen the framework resilience against various logical attacks.

CRedit authorship contribution statement

Ahmed Anas: Writing – review & editing, Writing – original draft.
Ayman A. Alhelbawy: Writing – review & editing. **Salwa El Gamal:**

Writing – review & editing. **Basheer Youssef:** Writing – review & editing.

References

- [1] Shahid J, Hameed MK, Javed IT, Qureshi KN, Ali M, Crespi N. A comparative study of web application security parameters: Current trends and future directions. *Appl Sci* 2022;12(8):4077.
- [2] Altulaihan EA, Alismail A, Frikha M. A survey on web application penetration testing. *Electronics* 2023;12(5):1229.
- [3] Felmetsger V, Cavedon L, Kruegel C, Vigna G. Toward automated detection of logic vulnerabilities in web applications. In: Proceedings of the USENIX security symposium, vol. 58, 2010, <http://dx.doi.org/10.5555/1929820.1929834>.
- [4] Kaur J, Garg U. State-of-the-art survey on web vulnerabilities, threat vectors, and countermeasures. In: Agrawal R, He J, Pilli ES, Kumar S, editors. *Cyber security in intelligent computing and communications*. Singapore: Springer; 2022, p. 3–17. http://dx.doi.org/10.1007/978-981-16-8012-0_1.
- [5] Deepa G, Thilagam P. Securing web applications from injection and logic vulnerabilities: Approaches and challenges. *Inf Softw Technol* 2016;74:160–80. <http://dx.doi.org/10.1016/j.infsof.2016.02.005>.
- [6] Doupé A, Cova M, Vigna G. Why johnny can't pentest: An analysis of black-box web vulnerability scanners. In: Detection of intrusions and malware, and vulnerability assessment: 7th international conference. 2010, p. 111–31. http://dx.doi.org/10.1007/978-3-642-14215-4_7.
- [7] Prandl S, Lazarescu M, Pham D-S. A study of web application firewall solutions. In: *Information systems security: 11th international conference, ICISS 2015, Kolkata, India, December 16–20, 2015. Proceedings 11*. Springer; 2015, p. 501–10.
- [8] OWASP. A01:2021 – broken access control. A01 broken access control - OWASP top 10:2021. 2022, https://owasp.org/Top10/A01_2021-Broken_Access_Control. [Accessed 19 June 2022].
- [9] Anas A, Elgamal S, Youssef B. Survey on detecting and preventing web application broken access control attacks. *Int J Electr Comput Eng (IJECE)* 2024;14(1):772–81.
- [10] Almushiti E, Zaki R, Thamer N, Alshaya R. An investigation of broken access control types, vulnerabilities, protection, and security. In: Shaikh A, Alghamdi A, Tan Q, Emary IMME, editors. *Advances in emerging information and communication technology*. Switzerland, Cham: Springer Nature; 2024, p. 253–69. http://dx.doi.org/10.1007/978-3-031-53237-5_16.
- [11] Zhong L. A survey of prevent and detect access control vulnerabilities. 2023, URL <https://arxiv.org/abs/2304.10600>, arXiv:2304.10600.
- [12] Ndiaye Y, Barais O, Blouin A, Bouabdallah A, Aillery N. Requirements for preventing logic flaws in the authentication procedure of web applications. In: Proceedings of the 34th ACM/SIGAPP symposium on applied computing. 2019, p. 1620–8. <http://dx.doi.org/10.1145/3297280.3297438>.
- [13] Scarfone K, Souppaya M, Dodson D. Secure Software Development Framework (SSDF) Version 1.1: Recommendations for Mitigating the Risk of Software Vulnerabilities. Technical Report NIST SP 800-218, Gaithersburg, MD: National Institute of Standards and Technology; 2022, <http://dx.doi.org/10.6028/NIST.SP.800-218>, URL <https://doi.org/10.6028/NIST.SP.800-218>.
- [14] Swamy N, Corcoran B, Hicks M. Fable: A language for enforcing user-defined security policies. In: Proceedings - IEEE symposium on security and privacy. 2008, p. 369–83. <http://dx.doi.org/10.1109/SP.2008.29>.
- [15] Chong S, Vikram K, Myers AC. SIF: Enforcing confidentiality and integrity in web applications. In: Proceedings of the 16th USENIX security symposium on USENIX security symposium. 2007, p. 1–16. <http://dx.doi.org/10.5555/1362903>.
- [16] Jia L, Vaughan JA, Mazurak K, Zhao J, Zarko L, Schorr J, et al. Aura: a programming language for authorization and audit. In: Proceedings of the 13th ACM SIGPLAN international conference on functional programming. 2008, p. 27–38. <http://dx.doi.org/10.1145/1411204.1411212>.
- [17] Vikram K, Prateek A, Livshits B. Ripley: automatically securing web 2.0 applications through replicated execution. In: Proceedings of the 16th ACM conference on computer and communications security. 2009, p. 173–86. <http://dx.doi.org/10.1145/1653662.1653685>.
- [18] Morgenstern J, Licata DR. Security-typed programming within dependently typed programming. In: Proceedings of the 15th ACM SIGPLAN international conference on functional programming. 2010, p. 169–80. <http://dx.doi.org/10.1145/1863543.1863569>.
- [19] Yip A, Wang X, Zeldovich N, Kaashoek MF. Improving application security with data flow assertions. In: Proceedings of the ACM SIGOPS 22nd symposium on operating systems principles. 2009, p. 291–304. <http://dx.doi.org/10.1145/1629575.1629604>.
- [20] Yang J, Hance T, Austin T, Solar-Lezama A, Flanagan C, Chong S. Precise, dynamic information flow for database-backed applications. In: Proceedings of the 38th ACM SIGPLAN conference on programming language design and implementation. 2016, p. 631–47. <http://dx.doi.org/10.1145/2908080.2908098>.
- [21] Zhang B, Li J, Ren J, Huang G. Efficiency and effectiveness of web application vulnerability detection approaches: A review. *ACM Comput Surv* 2021;54(5):190. <http://dx.doi.org/10.1145/3474553>.
- [22] Deepa G, Thilagam PS, Praseed A, Pais AR. DetLogic: A black-box approach for detecting logic vulnerabilities in web applications. *J Netw Comput Appl* 2018;109:89–109. <http://dx.doi.org/10.1016/j.jnca.2018.01.008>.
- [23] Li X, Xue Y. LogicScope: Automatic discovery of logic vulnerabilities within web applications. In: Proceedings of the 8th ACM SIGSAC symposium on information, computer and communications security. 2013, p. 481–6. <http://dx.doi.org/10.1145/2484313.2484375>.
- [24] Li X, Si X, Xue Y. Automated black-box detection of access control vulnerabilities in web applications. In: Proceedings of the 4th ACM conference on data and application security and privacy. 2014, p. 49–60. <http://dx.doi.org/10.1145/2557547.2557552>.
- [25] Pellegrino G, Balzarotti D. Toward black-box detection of logic flaws in web applications. In: Proceedings of the network and distributed system security symposium. San Diego, USA; 2014, <http://dx.doi.org/10.14722/ndss.2014.23021>.
- [26] Deepa G, Thilagam P, Khan F, Praseed A, Pais A, Palsetia N. Black-box detection of xquery injection and parameter tampering vulnerabilities in web applications. *Int J Inf Secur* 2018;17(1):105–19. <http://dx.doi.org/10.1007/s10207-016-0359-4>.
- [27] C. S, Jyostna G, P R, Eswari L. PROP - patronage of PHP web applications. *Int J Comput Sci Inf Technol* 2015;7:111–25. <http://dx.doi.org/10.5121/ijcsit.2015.7210>.
- [28] Monshizadeh M, Naldurg P, Venkatakrishnan VN. MACE: Detecting privilege escalation vulnerabilities in web applications. *ACM Trans Softw Eng Methodol* 2014;23(4):38:1–36. <http://dx.doi.org/10.1145/2660267.2660337>.
- [29] Son S, Shmatikov V, McKinley KS. FixMeUp: Repairing access-control bugs in web applications. In: Network and distributed system security symposium. Network and Distributed System Security Symposium (NDSS). 2013, URL <https://www.microsoft.com/en-us/research/publication/fixmeup-repairing-access-control-bugs-in-web-applications/>.
- [30] Ghorbanzadeh M, Shahriari HR. ANOVUL: Detection of logic vulnerabilities in annotated programs via data and control flow analysis. *IET Inf Secur* 2020;14(3):275–85. <http://dx.doi.org/10.1049/iet-ifs.2018.5615>.
- [31] Li X, Yan W, Xue Y. Sentinel: securing database from logic flaws in web applications. In: Proceedings of the second ACM conference on data and application security and privacy. 2012, p. 25–36. <http://dx.doi.org/10.1145/2133601.2133605>.
- [32] Marinescu P, Parry C, Pomarole M, Tian Y, Tague P, Papagiannis I. IVD: Automatic learning and enforcement of authorization rules in online social networks. In: 2017 IEEE symposium on security and privacy. 2017, p. 1094–109. <http://dx.doi.org/10.1109/SP.2017.33>.
- [33] Xing L, Chen Y, Wang X, Chen S. InteGuard: Toward automatic protection of third-party web service integrations. In: Proceedings of the 20th annual network and distributed system security symposium. San Diego, CA, USA; 2013, p. 1–10, URL <https://www.ndss-symposium.org/ndss2013/ndss-2013-programme/integuard-toward-automatic-protection-third-party-web-service-integrations>.
- [34] Gupta BB, Gupta S. PHP-sensor: A prototype method to discover workflow violation and XSS vulnerabilities in PHP web applications. In: Proceedings of the 12th ACM international conference on computing frontiers. Ischia, Italy; 2015, p. 59–66. <http://dx.doi.org/10.1145/2742854.2745719>, URL <https://dl.acm.org/doi/10.1145/2742854.2745719>.
- [35] Le M, Stavrou A, Kang B. DoubleGuard: Detecting intrusions in multitier web applications. *IEEE Trans Dependable Secure Comput* 2012;9(4):512–25. <http://dx.doi.org/10.1109/TDSC.2011.59>, URL <https://ieeexplore.ieee.org/document/6124490>.
- [36] Skrupsky N, Bisht P, Hinrichs T, Venkatakrishnan VN, Zuck L. TamperProof: a server-agnostic defense for parameter tampering attacks on web applications. In: Proceedings of the 3rd ACM conference on data and application security and privacy. 2013, p. 129–40. <http://dx.doi.org/10.1145/2435349.2435365>, URL <https://dl.acm.org/doi/10.1145/2435349.2435365>.
- [37] Li X, Xue Y. Block: a black-box approach for detection of state violation attacks towards web applications. In: Proceedings of the 27th annual computer security applications conference. 2011, p. 247–56. <http://dx.doi.org/10.1145/2076732.2076767>.
- [38] Cova M, Balzarotti D, Felmetsger V, Vigna G. Swaddler: An approach for the anomaly-based detection of state violations in web applications. In: Kruegel C, Lippmann R, Clark A, editors. *Recent advances in intrusion detection*. Berlin Heidelberg: Springer; 2007, p. 63–86, URL https://link.springer.com/chapter/10.1007/978-3-540-74320-0_4.
- [39] Provos N, Friedl M, Honeyman P. Preventing privilege escalation. In: 12th USENIX security symposium. 2003.
- [40] Ma L, Yan Y, Jiang H, Zhou Y. Analysis of privilege escalation based on hierarchical RBAC model. *J Phys: Conf Ser* 2020;1575(1):012060.
- [41] Liu Z, Zhao H, Li S, Li Q, Wei T, Wang Y. Privilege-escalation vulnerability discovery for large-scale RPC services: Principle, design, and deployment. In: Proceedings of the 2021 ACM Asia conference on computer and communications security. 2021, p. 565–77.
- [42] Ghawate M. Security incident response simulation project privilege escalation attack. 2024.

- [43] Deerwester S, Dumais ST, Furnas GW, Landauer TK, Harshman R. Indexing by latent semantic analysis. *J Am Soc Inf Sci* 1990;41(6):391–407.
- [44] Cortes C, Vapnik V. Support-vector networks. *Mach Learn* 1995;20:273–97.
- [45] Breiman L. Random forests. *Mach Learn* 2001;45:5–32.
- [46] Kotsiantis SB, Zaharakis I, Pintelas P, et al. Supervised machine learning: A review of classification techniques. *Emerg Artif Intell Appl Comput Eng* 2007;160(1):3–24.
- [47] Dahse J, Holz T. Static detection of logic bugs in web applications. In: *Proceedings of the 23rd USENIX security symposium*. USENIX Association; 2014, p. 361–76.